EV CHARGEOPS

Sprint 2 - Plano de Desenvolvimento

Backlog organizado por areas de Ciência da Computação para dividir o trabalho do grupo, desenvolver em paralelo e integrar o MVP de ponta a ponta.

5	2	1
frentes principais	modelos de IA	produto integrado

> **Objetivo deste documento**
> Transformar as decisões da Sprint 1 em trabalho executavel: quem faz o que, qual entrega depende de qual, como trabalhar em paralelo e o que precisa estar funcionando para considerar a Sprint 2 concluida.

Uso recomendado

- Definir um responsavel principal para cada frente.
- Abrir as tarefas como Issues no GitHub.
- Usar o contrato de sessão como interface comum entre todos.
- Integrar cedo e manter mocks para que nenhuma frente fique parada.

Navegação rápida

Use este mapa visual para localizar a frente desejada. O leitor de PDF também exibirá os capítulos no painel de marcadores.

1. Visão geral da Sprint 2 3
2. Mapa de dependências e trabalho em paralelo 4
3. Frente 1 - GoodWe, IoT e integração 5
4. Frente 2 - Backend e regras de aplicação 5
5. Frente 3 - Banco de dados e motor de rateio 6
6. Frente 4 - Inteligência Artificial e Ciência de Dados 8
7. Frente 5 - Frontend e IHC 8
8. DevOps, segurança e testes 9
9. Divisão sugerida para 5 pessoas 11
10. Ordem de execução recomendada 11
11. MVP e cortes de escopo 12
12. Checklist de conclusão 12
13. Decisões em aberto 13
14. Modelo de Issues no GitHub 13
15. Referência rápida por área de CC 14

Dica: para dividir o grupo, vá direto às seções 3 a 9. Para organizar a execução, use as seções 10 a 14.

1. Visao geral da Sprint 2

A Sprint 1 definiu o produto. A Sprint 2 precisa provar que o fluxo funciona em codigo: dados do carregador entram no sistema, sao ligados ao morador, validados, faturados e apresentados no aplicativo e no painel do síndico.

> **Pipeline que precisa existir**
> GoodWe/SEMS → Sessão → Reserva → Morador → Validação/Anomalia → Rateio → Fatura → App e Painel do Síndico

Frente	Area de CC	Responsabilidade principal
1. GoodWe / IoT	Redes, IoT, Sistemas Distribuidos	Fazer os dados do carregador chegarem ao sistema e criar a rota de contingencia.
2. Backend / API	Engenharia de Software	Usuários, reservas, sessões, atribuição, autenticação e orquestracao.
3. Banco + Rateio	Banco de Dados, Algoritmos	Modelar dados, garantir integridade e calcular cobranças auditaveis.
4. IA / Dados	IA, Machine Learning, Ciência de Dados	Detectar anomalias e recomendar melhores horários.
5. Frontend / IHC	IHC, Desenvolvimento Web	PWA do morador e painel administrativo do síndico.

Principios para dividir o trabalho

- Cada pessoa tem uma frente principal, mas o produto final e compartilhado.
- Nenhuma frente deve depender da API real para comecar: usem mocks do mesmo contrato.
- Integram-se pequenas partes continuamente, em vez de deixar a integração para o final.
- Tarefas financeiras, de atribuição de sessão e de anomalia devem ter testes automatizados.
- As decisões ainda não comprovadas na Sprint 1 devem virar tarefas de validação técnica, não suposicoes.

2. Mapa de dependencias e trabalho em paralelo



Figura 1 - O contrato de sessão e a interface comum entre as cinco frentes.

O artefato que destrava o grupo: SessionDTO

Antes de cada pessoa criar seu próprio formato, definam um objeto padrao de sessão. Mesmo que a integração GoodWe ainda não esteja pronta, todos desenvolvem contra este contrato.

```
{
  "session_id": "abc123",
  "charger_id": "HCA01",
  "card_id": "RFID-05",
  "started_at": "2026-09-22T19:00:00",
  "ended_at": "2026-09-22T21:10:00",
  "total_kwh": 14.8,
  "green_kwh": 4.2,
  "grid_kwh": 10.6,
  "average_power_kw": 6.8,
  "max_power_kw": 7.1,
  "status": "completed"
}
```

Regra de ouro

Pessoa 1 converte SEMS/Modbus para o contrato. Pessoas 2, 3, 4 e 5 usam o mesmo contrato em mocks. Quando a fonte real estiver pronta, troca-se o mock sem redesenhar o restante do sistema.

3. Frente 1 - GoodWe, IoT e integração

Area: Redes, IoT e Sistemas Distribuidos. Esta frente e a primeira investigacao crítica porque define quais dados reais existem e como entram no EV ChargeOps.

Tarefa	Entregavel / pronto quando
Validar autenticação SEMS+	Existe uma chamada reproduzivel, documentada, que autentica sem credenciais hardcoded no codigo.
Mapear endpoints e campos	Documento interno com endpoints, parametros, respostas e campos realmente disponíveis.
Capturar sessão real	JSON real anonimizado salvo como fixture de desenvolvimento.
Criar cliente SEMS+	Modulo Python consulta sessões, trata erros e normaliza para SessionDTO.
Validar Green / Grid / Charged	Fica comprovado de onde vem cada campo e o significado de cada unidade.
Investigar Modbus TCP	Lista de registradores/leituras realmente acessiveis no HCA G2 ou decisão documentada de que não e viável.
Criar simulador HCA G2	Simulador gera sessões e telemetria suficientes para backend e testes.
Gateway Raspberry Pi	Servico pymodbus le dados, persiste localmente e envia ao backend.
Buffer SQLite	Queda de internet não perde sessões; reenvio acontece quando a conexao retorna.
Reconhecer duplicidade	Gateway e SEMS não criam duas sessões independentes para o mesmo evento.

Entregaveis recomendados

- docs/goodwe-data-contract.md
- fixtures/goodwe/session_real.json
- backend/integrations/goodwe/sems_client.py
- simulator/
- gateway/ com pymodbus + SQLite

> **Validação crítica da Sprint 1**
> Não assumir que todo campo descrito na pesquisa estará disponível pela API ou por Modbus exatamente como imaginado. SEMS+ e Modbus precisam ser tratados como hipoteses tecnicas até serem comprovados na pratica.

4. Frente 2 - Backend e regras de aplicação

Area: Engenharia de Software. O backend e o cerebro que conecta integração, banco, rateio, IA e interfaces.

Modulo	O que precisa fazer
Autenticação	Login, perfis e autorização de morador versus gestor.
Usuários e unidades	Cadastrar moradores, apartamentos/unidades e papeis.
Carregadores	Cadastrar equipamentos e estado operacional básico.

Modulo	O que precisa fazer
Reservas	Criar, cancelar, listar e impedir conflito de horário.
No-show	Cancelar reserva apos 20 min sem sessão e registrar reincidencia.
Sessões	Receber SessionDTO, persistir e expor histórico.
Atribuição	Vincular sessão a reserva/morador pelo intervalo de tempo.
Tarifas	CRUD de faixas, precos, ociosidade, imposto e overhead.
Faturamento	Acionar o motor de rateio e gerar faturas mensais.
Anomalias	Chamar validacoes/IA e encaminhar suspeitas para revisao.
Recomendação	Expor endpoint com melhor janela e custo previsto.
Relatórios	Fornecer dados de consumo, arrecadacao e auditoria ao painel.

Fluxo principal do backend

1. Receber ou consultar uma sessão do carregador.
2. Normalizar para SessionDTO.
3. Conciliar SEMS e gateway quando houver duas fontes.
4. Encontrar reserva/morador correspondente.
5. Executar validacoes fisicas e detector de anomalias.
6. Se normal, enviar ao motor de rateio; se suspeita, suspender para revisao.
7. Persistir memoria de cálculo e atualizar fatura.
8. Expor resultado ao app do morador e painel do gestor.

5. Frente 3 - Banco de dados e motor de rateio

Areas: Banco de Dados e Algoritmos. Esta frente protege a consistencia financeira do sistema e transforma sessões em cobranças auditaveis.

Modelo de dados mínimo

Entidade	Papel
User	Morador ou gestor.
CondoUnit	Apartamento/unidade condominial.
Charger	Carregador físico.
Reservation	Janela reservada por um usuário.
ChargingSession	Sessão de recarga normalizada.
Tariff	Faixas e precos configuraveis.
Invoice	Fatura mensal do usuário.
InvoiceItem	Parcela/sessão que compoe a fatura.
Anomaly	Sessão suspeita e decisão de revisao.
AuditEvent	Trilha de alteracoes e reconciliação.

Regras de integridade importantes

- Uma sessão não pode ser cobrada para dois moradores ao mesmo tempo.
- Reservas do mesmo carregador não podem se sobrepor alem da capacidade configurada.
- Tarifas devem ser versionadas ou manter validade temporal para que uma fatura antiga continue reproduzivel.
- Fatura fechada não deve mudar silenciosamente quando uma configuração atual for alterada.
- Toda revisao manual de anomalia deve deixar evento de auditoria.

Formula de rateio

> **Composição da cobrança**
> Energia Green + Energia Grid + Ociosidade = Subtotal. Sobre o subtotal entram Imposto e Overhead. A fatura mensal soma as sessões validas do morador.

Parcela	Regra
Energia Green	kWh de origem solar x tarifa Green aplicavel.
Energia Grid	kWh da rede x tarifa Grid aplicavel.
Ociosidade	Minutos depois da tolerância de 15 min x taxa por minuto.
Imposto	Percentual parametrizado para o período/local.
Overhead	Percentual operacional definido pelo condominio, inclusive 0%.

Caso que precisa ser tratado na Sprint 2: sessão atravessando tarifas

Se a recarga comeca em uma faixa e termina em outra, a cobrança deve ser particionada por intervalo, desde que a telemetria permita estimar o consumo em cada trecho. Não use simplesmente a tarifa do horário inicial para toda a sessão.

> **Exemplo**
> 17:30-18:00 = faixa intermediaria; 18:00-20:30 = ponta. O valor final e a soma do consumo atribuido a cada faixa vezes sua respectiva tarifa.

6. Frente 4 - Inteligência Artificial e Ciência de Dados

A Sprint 1 definiu duas aplicações de IA: detectar sessões anormais antes da cobrança e recomendar a janela de recarga mais vantajosa.

IA A - Detecção de anomalias

Item	Implementacao
Entrada	kWh, duracao, potencia media/máxima, kWh/h, horário, carregador, Card ID e curva de potencia quando houver.
Regras deterministicas	Antes do ML, bloquear inconsistencias fisicamente impossiveis ou campos invalidos.
Modelos	Isolation Forest e Local Outlier Factor (LOF), conforme Sprint 1.
Saída	Score + classificacao normal/suspeita + sinais que ajudaram a revisar.
Ação	Sessão suspeita não entra automaticamente na fatura; vai para fila do gestor.
Dados iniciais	Dataset público para prototipo, validando distribuicao antes de misturar com dados reais FIAP.

> **Boa separacao de responsabilidades**
> Uma sessão impossivel para a potencia do carregador não precisa de IA. Regras fisicas pegam erros obvios; o modelo fica responsavel por padroes menos evidentes.

IA B - Recomendação de melhor horário

Componente	O que usar
Features	Hora, dia da semana, histórico, tarifas, geração solar, consumo do predio, clima e SOC quando realmente disponível.
Baseline	Regressao linear com features temporais/sazonais.
Evolução	XGBoost/Gradient Boosting se houver ganho demonstravel.
Horizonte	Recomendação nas próximas 24-48h conforme dados disponíveis.
Saída	Horário recomendado, custo previsto e comparação com carregar agora.
Validação	Back-test temporal; não misturar futuro no treino.

7. Frente 5 - Frontend e IHC

Area: Interacao Humano-Computador e Desenvolvimento Web. Existem duas experiencias sobre a mesma API: morador e síndico.

PWA do morador

Tela	Objetivo
Login	Autenticar e identificar o perfil.

Tela	Objetivo
Home	Próxima reserva, consumo recente e atalhos.
Reservar	Mostrar carregadores/janelas disponíveis e criar reserva.
Recomendação	Comparar carregar agora versus janelas sugeridas.
Sessões	Histórico de recargas e status.
Fatura	Valor mensal e composição.
Detalhe	Memoria de cálculo por sessão/parcela.
Perfil	Dados basicos do morador e unidade.

Painel do síndico

Tela	Objetivo
Dashboard	Situação geral do sistema.
Carregadores	Estado operacional e sessões atuais.
Reservas	Agenda e conflitos.
Sessões	Histórico completo.
Anomalias	Revisar, aprovar ou rejeitar sessões suspeitas.
Tarifas	Configurar faixas, Green, Grid e ociosidade.
Configurações	Imposto, overhead e parametros.
Financeiro	Arrecadacao mensal e relatórios.

> **Criterio de UX**
> O morador precisa entender o que pagar e por que; o síndico precisa enxergar excecoes e configurações sem editar codigo.

8. Responsabilidades compartilhadas: DevOps, segurança e testes

Estas atividades atravessam todas as frentes. Um integrante pode coordenar, mas não devem ficar isoladas em uma sexta pessoa inexistente.

Area	Mínimo esperado
Docker Compose	Subir PostgreSQL, backend, frontend e simulador de forma reproduzivel.
CI/CD	GitHub Actions executa testes e lint em pull requests.
Configuração	Segredos fora do repositorio; .env.example documentado.
Testes backend	pytest para reservas, atribuição, rateio, tarifas e reconciliação.
Testes de IA	Dataset de teste fixo, métricas e exemplos suspeitos/saudaveis.

Area	Mínimo esperado
Testes de integração	Fluxo completo com sessão simulada até a fatura.
Segurança	Autorização por perfil e validação de entradas.
Auditoria	Alteracoes financeiras e decisões de anomalia rastreaveis.
Documentacao	README de execução + contrato GoodWe + endpoints principais.

Casos de teste que não podem faltar

- Impedir duas reservas incompatíveis para o mesmo carregador.
- Atribuir uma sessão exatamente ao morador da reserva correta.
- Calcular corretamente Green, Grid, ociosidade, imposto e overhead.
- Dividir uma sessão que cruza faixas tarifarias, quando houver dados suficientes.
- Aplicar tolerância de 15 min antes de ociosidade.
- Cancelar no-show apos 20 min e registrar reincidencia.
- Bloquear sessão fisicamente impossivel antes da fatura.
- Conciliar SEMS e gateway sem duplicar cobrança.

9. Divisão sugerida para um grupo de 5 pessoas

A distribuicao abaixo separa areas tecnicas, mas mantem interfaces claras. Troquem as pessoas conforme a afinidade do grupo.

Pessoa	Frente principal	Entregas principais	Apoio / interface
Pessoa 1	GoodWe / IoT	SEMS+, contrato de dados, simulador, gateway, SQLite	Alinha SessionDTO com Backend e IA.
Pessoa 2	Backend / API	Auth, reservas, sessões, atribuição, endpoints	Integra todas as frentes e define contratos REST.
Pessoa 3	Banco + Rateio	Schema, migrations, tarifas, faturas, memoria de cálculo	Cria testes financeiros e regras de integridade.
Pessoa 4	IA / Dados	Anomalias, regras fisicas, recomendação de horário	Define features com Pessoa 1 e endpoints com Pessoa 2.
Pessoa 5	Frontend / IHC	PWA morador, painel síndico, estados de erro/loading	Trabalha com API mockada e depois integra backend.

Como evitar bloqueios entre as pessoas

Se esta parte ainda não existe...	O restante do grupo faz...
SEMS+ real	Usa fixture JSON / SessionDTO mockado.
Banco final	Usa migrations iniciais e contratos Pydantic.
IA treinada	Backend retorna score/recomendação mockados com o mesmo schema.
Frontend final	Endpoints sao testados por Swagger/Postman/pytest.
Gateway Raspberry	Simulador envia os mesmos eventos REST esperados.

10. Ordem de execução recomendada

A ordem abaixo distingue investigacao crítica de implementacao. A integração GoodWe deve ser investigada no primeiro dia, mas o produto não deve ficar parado esperando sua conclusão.

Etapa	Prioridade	Resultado
1. Contrato de dados + schema inicial	P0	SessionDTO, fixture e entidades principais alinhadas.
2. Backend base + ambiente Docker	P0	API sobe com Postgres e dados mockados.
3. Reservas + sessões + atribuição	P0	Fluxo operacional central funciona.
4. Motor de rateio + testes	P0	Sessão normal gera memoria de cálculo e fatura.
5. Frontend básico	P0	Morador reserva/consulta; síndico enxerga sessões.
6. Anomalias	P0/P1	Sessão suspeita e suspensa para revisao.
7. Recomendação de horário	P1	Endpoint e tela com custo previsto.
8. Integração SEMS real	P0	Substitui fixture pela fonte real validada.

Etapa	Prioridade	Resultado
9. Gateway/Modbus	P1	Contingencia demonstrada com simulador e, se possível, hardware.
10. Integração final + pitch	P0	Demo ponta a ponta reproduzivel e documentada.

11. MVP: o que preservar se o tempo apertar

> **Fluxo mínimo obrigatorio**
> Usuário → Reserva → Sessão → Atribuição → Detecção/validação → Rateio → Fatura → Frontend

Dentro do MVP

- Reserva de janela.
- Sessão de recarga real ou simulada no mesmo contrato.
- Atribuição da sessão ao usuário.
- Rateio Green/Grid + ociosidade + imposto + overhead.
- Memoria de cálculo auditavel.
- Detecção de anomalia com revisao do síndico.
- Recomendação de melhor horário em versão funcional.
- PWA do morador e painel mínimo do gestor.

Não gastar tempo agora

- OCPP.
- PIX/cartao e gateway de pagamentos.
- Multi-tenancy para varios condominios.
- Aplicativo Android/iOS nativo.
- Chatbot/NLP.
- Design extremamente sofisticado antes do fluxo funcionar.
- Certificacao metrologica fiscal como produto acabado.

12. Checklist de conclusão da Sprint 2

	Item de verificacao
☐	API sobe localmente com um unico comando/documentacao clara.
☐	PostgreSQL e migrations funcionam do zero.
☐	Existe contrato de sessão documentado e fixture de teste.
☐	Existe prova de acesso aos dados reais GoodWe ou limitacao documentada.
☐	Morador consegue criar uma reserva valida.
☐	Sessão e atribuida ao morador correto.
☐	Fatura exibe Green, Grid, ociosidade, imposto e overhead.
☐	Memoria de cálculo permite reproduzir o valor.
☐	Sessão fisicamente impossivel e bloqueada.
☐	Anomalia aparece na fila do síndico e pode ser revisada.

	Item de verificacao
☐	Existe recomendação de horário com custo previsto.
☐	Frontend do morador cobre reserva, histórico e fatura.
☐	Painel do síndico cobre sessões, anomalias, tarifas e relatório básico.
☐	Testes principais rodam no CI.
☐	Simulador permite demonstrar o fluxo sem depender do laboratorio.
☐	README explica como executar e demonstrar.
☐	Pitch/demo possui roteiro e dados de contingencia offline.

13. Decisões que precisam ser fechadas cedo

Estas não devem ficar escondidas até a semana final. Transformem cada uma em Issue de validação.

Questão	Decisão/ação necessaria
SEMS+	Como autenticar? Quais endpoints sao oficialmente utilizaveis? Quais campos existem de verdade?
Modbus	O HCA G2 fornece os dados necessarios via Modbus TCP? Quais registradores?
Green/Grid por faixa	A telemetria tem granularidade suficiente para dividir consumo quando a sessão atravessa tarifas?
RFID compartilhado	O que fazer com sessão sem reserva? Evitar atribuir automaticamente a um suposto dono de cartao compartilhado.
Fonte da verdade	Definir precedencia SEMS/gateway por campo, timestamp, duplicidade e divergencia.
Impostos	No MVP academico, quais parametros serao demonstrados sem transformar o sistema em um motor tributario completo?
Dados de IA	Quais campos do dataset público realmente combinam com a distribuicao do laboratorio?

14. Como transformar isto em Issues no GitHub

Cada tarefa relevante pode usar o mesmo molde. Isso deixa a divisão do grupo transparente e reduz discussoes sobre quando algo esta realmente pronto.

Campo	Exemplo
Titulo	[RATEIO] Calcular ociosidade apos tolerância de 15 minutos
Contexto	A Sprint 1 define taxa de overstay apos 15 min do fim da carga.
Entrada	Sessão, fim de fluxo de energia, fim de ocupacao, tarifa/min.
Saída	idle_minutes, idle_cost e memoria de cálculo.
Criterios de aceite	14 min = R$0; 15 min = R$0; 16 min = 1 min cobrado; falha do sistema = R$0.
Dependencias	Schema ChargingSession + Tariff.

Campo	Exemplo
Testes	Casos unitarios e integração com invoice.
Responsavel	Pessoa 3.
Labels	sprint-2, rateio, P0

15. Referencia rápida: Ciência da Computação no projeto

Area de CC	Aplicação no EV ChargeOps
Engenharia de Software	Arquitetura, APIs, regras, contratos, manutencao.
Banco de Dados	PostgreSQL, integridade, transacoes, auditoria.
Redes	HTTP, SEMS, conectividade e Modbus TCP.
Sistemas Distribuidos	Nuvem + gateway local + reconciliação.
IoT / Embarcados	Raspberry Pi e interacao com carregador.
Algoritmos	Rateio, janelas, conflitos e regras.
IA / ML	Outliers e previsão/recomendação.
Ciência de Dados	Features, datasets, validação temporal.
IHC	Experiencia do morador e do síndico.
Segurança	Autenticação, autorização, segredos e LGPD.
DevOps	Docker, CI e reproducibilidade.
Testes	Confiabilidade do fluxo e do cálculo financeiro.

> **Resumo em uma frase**
> O grupo deve implementar um unico pipeline integrado: coletar uma sessão GoodWe, descobrir de quem e, verificar se faz sentido, calcular quanto custa e explicar esse resultado claramente ao morador e ao síndico.